

GRIDLOCK-FREE BENGALURU

Predictive Parking Intelligence, Anomaly Alerts & What-If Dispatch Simulator

Bengaluru Traffic Police (BTP) Parking Violation Management Solution

Solo Innovator | Dataset: 298,450 Records (Nov 2023 - Apr 2024) | Release: June 2026

The Problem & The SCITA Enforcement Leakage

Systemic Enforcement Leakage

Raw violation analysis identified thousands of approved parking violations that were never sent to the SCITA challan system.

Leakage Formula: `SCITA Gap = (validation_status == 'approved') & (data_sent_to_scita == False)`.

Consequence: Repeat parking offenders escape penalties, which directly undermines traffic rules and traffic flow.

Illegal Parking Gridlock

Reactive parking management fails to prevent blockages at critical junctions.

Top zones include dense commercial sectors with high traffic volume (Shivajinagar, Upparpet, City Market).

Audit & Target Prioritization

Audit metrics pinpoint the exact station jurisdictions suffering from communication failures (SCITA Gap Count).

The pipeline prioritizes cells with large SCITA gaps to restore BTP's enforcement credibility.

Manpower Allocation Solution

Rather than uniform deployment, BTP can target grid cells by priority to maximize enforcement coverage.

Coordinates patrols to shift windows (Morning, Evening, Night) based on local violation patterns.

7-Layer Upgraded Pipeline Architecture

Layer 1: Preprocessing & Cleaning

Drops unused columns, converts date timestamps to Indian Standard Time (IST), and parses JSON violation types.

Layer 2: Features & Spillover Index

Establishes dual spatial grids: 3-decimal (~110m cells) for scoring and 2-decimal (~1.1km cells) for temporal forecast. Computes junction multipliers and Congestion Spillover indices.

Layer 3: Unsupervised & Supervised Models

Runs DBSCAN spatial clustering, XGBoost temporal regressor, and Isolation Forest anomaly flagging.

Layer 4: Composite Risk Scoring

Safe MinMax normalization of density, severity, junctions, peak hours, and weights scaled by Congestion Spillover.

Layer 5: Scheduler & Vehicle Asset Optimization

Groups top 50 cells, pivots shifts, and assigns optimized patrol vehicles (Tows, Jeeps, Bikes) based on weight mode.

Layer 6: Dynamic Dashboard with What-If Simulator

Renders Map, Plotly charts, and unified dashboard incorporating dynamic JS re-ranking and Leaflet opacity filters.

Layer 7: Outbox Email Dispatch Center

Simulates emails to police station heads, queued and dispatched 1 hour before shift, synced to timezone clock.

Automated Verification Suite

Runs `verify_pipeline.py` to assert schema fields, output file existence, and enforce design constraints.

Unsupervised & Supervised Machine Learning Suite

Model A: Spatial Hotspot Clusterer (DBSCAN)

Filtered to approved records to find dense spatial violation zones.

Metric: Haversine distance on coordinates (eps=300m, min_samples=10).

Detected 175 major spatial hotspot clusters.

Performance Cache: DBSCAN labels are cached in dbscan_labels.npy to make dashboard load times instant on re-execution.

Model B: Temporal Density Predictor (XGBoost)

Forecasts next-hour violation counts to serve as a traffic congestion proxy.

Inputs: Chronologically sorted dataset with 1h and 3h rolling lags computed using pandas transform.

Validation: Evaluated via TimeSeriesSplit (3 folds).

Metrics: Achieved validation RMSE of 26.08 and MAE of 11.06.

Model C: Spatial-Temporal Anomaly Detector

Algorithm: sklearn.ensemble.IsolationForest trained on cell-hour aggregates.

Training Features: Density, average severity, average vehicle weight, scita gap count, and peak-hour ratio.

Parameters: Fit with 5% contamination rate.

Flagged 786 anomalous cell-hour combinations.

Actionable Output: Mapped back to top 50 cells under is_anomaly. Anomalies blink distinctly on Leaflet map markers to catch operators' eyes immediately.

Alerts Feed: Dedicated sidebar panel lists anomalous zones in real-time.

Advanced Intelligence & Optimization

Congestion Spillover Index

Formulated to inflate risk scores of cells near major intersections to prevent traffic gridlock.

Algorithm: KDTREE neighborhood search queries surrounding cells within a 300m bounding box (0.003 degrees).

Formula: Spillover Index = $1.0 + 0.15 \times$

$\log_{10}(\text{neighboring_junction_violations})$.

Scoring Fusion: Base Risk Score is multiplied by the Spillover Index, prioritizing zones adjacent to high-risk junctions.

Vehicle-Type Patrol Optimization

Classifies grid cells and optimizes vehicle assignments based on local vehicle weight distributions (mode).

Heavy Tow Truck: For heavy mode grids (Tankers, Buses, Cabs).

Patrol Jeep: For passenger grids (Cars, Autos).

Interceptor Bike: For light grids (Motorcycles, Scooters).

Composite Risk Scoring Fusion

Evaluates cell priority using safe minmax normalization:

$$\text{Risk} = \text{Base_Risk} \times \text{Spillover_Index}$$

$$\text{Base_Risk} = 40\% \text{ Density} + 25\% \text{ Severity} + 20\% \text{ Junction} + 10\% \text{ Peak Hour} + 5\% \text{ Vehicle Weight}$$

Time-Synced Notifier System

Groups the schedule by station and shift to send targeted dispatch briefs.

Simulated email client tab in dashboard updates email status badges (Dispatched, Queued, Archived) dynamically based on the current timezone clock.

Emails contain detailed briefings: ranks, coordinates, expected hourly breakdown, and assigned patrol asset (Tow, Jeep, Bike).

What-If Dispatch Simulator & Front-End UI

Interactive Sliders

Dispatchers can dynamically adjust available patrol units (5-50) and priority bias (Risk vs. Density) via sidebar sliders.

Score Blending: JavaScript blends normalized risk and density on slider change:

$$\text{BlendedScore} = \text{Bias} * \text{Risk} + (1 - \text{Bias}) * \text{Density}$$

Violation Risk Mitigation Score (VRMS)

Computes a real-time mitigation score in JS to measure the percentage of total risk covered by dispatched patrols:

$$\text{VRMS} = (\text{Sum of Blended Scores of Dispatched Cells} / \text{Sum of Blended Scores of All 50 Cells}) * 100$$

Rendered in a glowing neon progress bar that updates instantly.

Leaflet Map Integration

Dispatched patrol units remain at high opacity (0.8) and display rank-coded colors.

Standby units are dynamically faded to low opacity (0.15) to focus dispatchers' attention.

Aesthetic UI/UX Enhancements

Floating Map Overlay Switcher: Programmatically handles layer swapping (Static Heatmap vs Live Dispatch Heatmap) floated directly on the map card.

Anomaly Alerts Feed: Dedicated sidebar card shows live warnings.

Emoji Removal: Emojis are completely removed from all dashboard components and map controls for a professional look.

Verification & Packaged Deliverables

Automated Verification Suite

Executing `verify_pipeline.py` runs the pipeline, checks input data, and runs structural asserts.

Schema Verification: Asserts that `patrol_schedule.csv` contains upgraded columns: `is_anomaly`, `spillover_index`, and `assigned_patrol_unit`.

Email Verification: Asserts that `simulated_emails.log` contains patrol vehicle assignments.

Output Checks: Asserts `map.html`, `charts.html`, `dashboard.html`, `index.html`, and `patrol_schedule.pdf` exist and are non-empty.

BTP Enforcement Deliverables Archive

Deliverables are zipped into `BTP_Enforcement_Pipeline.zip`.
Design Bound Exclusions: Excludes large raw violations.csv (109MB), local `index.html`, and dbscan cache labels (`dbscan_labels.npy`) to ensure clean file transfer.

Packaged Files:

`run_pipeline.py` & `generate_pdf.py`
`verify_pipeline.py`
`dashboard.html`, `map.html` & `charts.html`
`patrol_schedule.pdf` & `patrol_schedule.csv`
`simulated_emails.log`